CAPÍTULO 1

Material y Software

1.1 Arduino

Arduino es una herramienta/plataforma de código abierto basada en una placa con un micro-controlador de Atmel que puede obtener información del entorno a través de sensores, e influir sobre el mediante actuadores conectados a los pines de salida de la placa. Programado mediante el lenguaje de programación de arduino que estudiaremos mas adelante.

Hay una gran cantidad de placas oficiales arduino, que integran distintos tipos de procesadores, comunicaciones, E/S, etc. En el presente manual, para mostrar el uso de dispositivos y software arduino, usaremos la placa "Arduino UNO" que se muestra en la Figura 1.1.- Arduino UNO y cable USB, cuyas características principales, se especifican en la Tabla 1.1.

Figura 1.1.- Arduino UNO y cable USB

Los dispositivos arduino tienen una serie de conectores divididos en tres bloques, a través de los cuales podrán interaccionar con el mundo exterior: Pines de potencia, Entradas/salidas digitales y salidas analógicas.

Microcontrolador	ATmega328
Voltaje Operación	5V
Voltaje Entrada (Recomendado)	7-12V
Voltaje Entrada (Límite)	6-20V
Entrada/Salidas Digitales	14 (6 proporcionan salidas PWM)
Entradas Analógicas	6
Corriente DC por pin E/S	40 mA
Corriente DC pin 3.3V	50 mA
Memoria Flash	32 KB (Atmega328), 0.5 KB usado por el bootloader
SRAM	2 KB (ATmega328)
EEPROM	1 KB (ATmega328)
Frecuencia Reloj	16 MHz

Tabla 1.1.- Especificaciones técnicas Arduino UNO

1.2 Conectores Arduino UNO

Los pines del procesador usado en las placas Arduino están disponibles para expandir las posibilidades de nuestra placa mediante las denominadas placas “Shield”, o la conexión directa de dispositivos, sensores, LED's, etc. Estos pines en Arduino UNO, se distribuyen en tres tipos de conectores: Potencia, Entradas/salidas digitales y salidas analógicas.

1.2.1 Entradas y salidas de potencia

Las placas arduino pueden ser alimentadas a través del puerto USB o mediante una fuente de alimentación externa (se selecciona de forma automática) y generar valores estables de tensión que pueden ser usados por dispositivos externos conectados a la placa. En la figura 1.2 resaltamos el conector usado para las entradas/salidas de potencia.

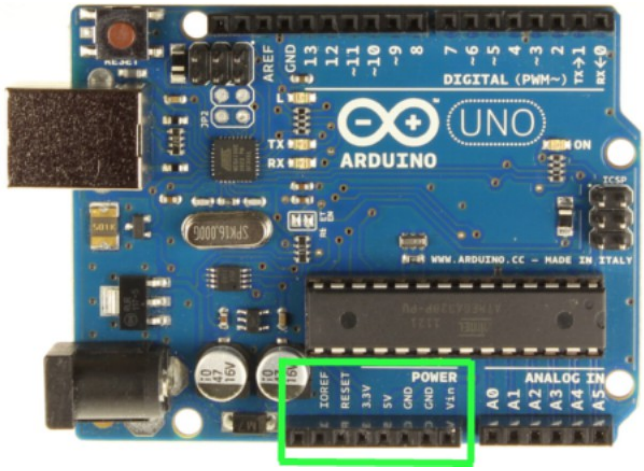

Figura 1.2. Conector de potencia

Donde los pines de potencia son:

- **Vin**. Cuando usamos una fuente de alimentación externa, Vin es usado como voltaje de entrada.
- **3.3V.** Voltaje generado por la placa para ser usado por dispositivos/sensores externo. Máxima corriente suministrada es 50 mA. La placa puede ser alimentada desde el jack de potencia con una tensión DC entre 7 y 12 V, el conecto USB o a través del pin VIN con una tensión de entre 7 y 12 V. Si alimentamos la placa a través de los pines 3.3V o 5V, podemos dañar la placa
- **GND.** Pines de tierra.
- **5V.** Voltaje generado por la placa Arduino para ser usado por dispositivos/sensores externos.
- **IOREF.** Voltaje de referencia a cual opera el microcontrolador.

1.2.2 Entradas Salidas Digitales

El conector de E/S dispone de 14 pines como se muestra en la figura 1.3. Cada uno de ellos opera a 5 V y un máximo de 40 mA, disponen de una resistencia pull-up (desconectado por defecto) de 20-50 kΩ. Algunos de estos pines tienen una función especializada:

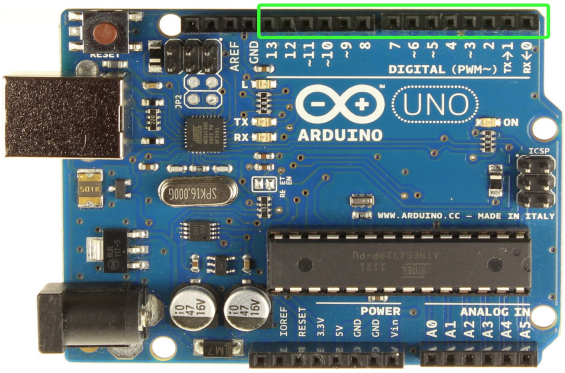

Figura 1.3. Conector E/S Digitales

- **Serie: Pin 0 (RX) y 1 (TX).** Usados para recibir (RX) y transmitir (TX) datos vía serie. Estos pines están conectados a los correspondientes pines del chip serie USB-to-TTL Atmega8U2.
- **Interrupciones externas: Pines 2 y 3.** Estos pines pueden ser configurados para producir interrupciones externas y que puede ser configurada para que se produzca a nivel bajo, cambio de nivel (nivel), umbral de subida o bajada (flanco).
- **PWM: Pines 3, 5, 6, 9, 10 y 11.** Proporcionan un sistema de modulación por ancho de pulsos de 8 bits, usando la librería arduino **analogWrite()**.
- **SPI: Pines 10(SS), 11(MOSI), 12(MISO) y 13(SCK).** Estos pines soportan comunicaciones vía bus SPI usando la librería arduino SPI.
- **LED: Pin 13.** Hay un diodo LED conectado a este pin. Cuando su valor es alto, el LED está encendido y apagado cuando el valor es bajo.

Todos los pines pueden ser usados de forma individual simplemente como pines digitales de entrada o salida.

1.2.3 Entradas Analógicas

Arduino UNO dispone de 6 entradas analógicas (Figura 1.4), etiquetadas desde A0 a A5, cada una de las cuales con 10 bits de resolución. Por defecto, su tensión de referencia es de 0-5 Volts., aunque es posible cambiar estos valores usando el pin AREF y la función de la librería arduino **analogReference()**.

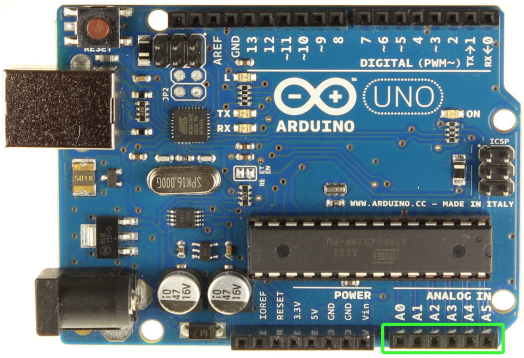

Figura 1.4: Entradas analógicas

Ademas, algunos pines analógicos tienen una función particular:

- **TWI (Two Wire Interface o I2C Interface)**: los pines **A4** o **SDA** y **A5** o **SCL** soportan la comunicación TWI usando la librería arduino Wire.

Otro par de pines con funciones especiales:

- **AREF.** Voltaje de referencia para las entradas analógicas
- **RESET.** Usada para producir una iniciación de la placa arduino.

1.3 Programación Arduino. IDE Arduino

Para trabajar con el entorno arduino lo primero que debemos decidir es la tarjeta arduino que vamos a utilizar y el sistema operativo en el que vamos a trabajar. Con anterioridad, ya hemos indicado que en el ámbito de este manual usaremos la placa **Arduino UNO** (Figura 1.1), mientras que el sistema operativo usado será **WINDOWS**.

Como herramienta de desarrollo software usaremos un entorno IDE (Integrated Development Environment) que nos ayude a escribir nuestros programas y a descargarlos en la placa arduino. En esta documentación estudiaremos dos entornos IDE distintos que podrán ser usados independientemente y según los gustos y características particulares del lector, así como la complejidad del software a ser desarrollado. A saber, el entorno de programación Arduino y Eclipse.

1.3.1.1 Instalación Entorno Arduino (IDE Arduino)

En primer lugar, descargamos el entorno “Arduino” el cual está escrito en Java, aunque el lenguaje de programación usado es C++. La última versión del software arduino puede ser descargado en http://www.arduino.cc/en/Main/Software. Actualmente (20-09-2013), la última versión disponible es **Arduino 1.0.5.**

Si la placa arduino usada dispone de un conector USB, se necesitará instalar los controladores (drivers) para el chip FTDI de la placa. Los Puedes encontrar en el directorio *drivers/FTDI USB Drivers* de la versión arduino instalada. Al conectar la placa, Windows mostrará un asistente para añadir un nuevo hardware para estos drivers. Podrás encontrar la última versión de los drivers en http://www.ftdichip.com/Drivers/VCP.htm.

El proyecto **arduino-erw (http://code.google.com/p/arduino-erw/)** incluye una pequeña modificación del entorno original, incluyéndose tanto el entorno IDE arduino como los drivers necesarios para interaccionar con la placa, para ser instalado de forma automática.

Para nuestros propósitos, no centraremos inicialmente en la instalación del entorno arduino (Arduino-1.0.5-windows.exe) que obtendremos de http://www.arduino.cc/en/Main/Software. Para ello se hace Dbl-Click sobre el archivo y aparecerá la Figura 1.5

Figura 1.5

se pulsa "I Agree" y obtendremos la imagen que se muestra en la Figura 1.6

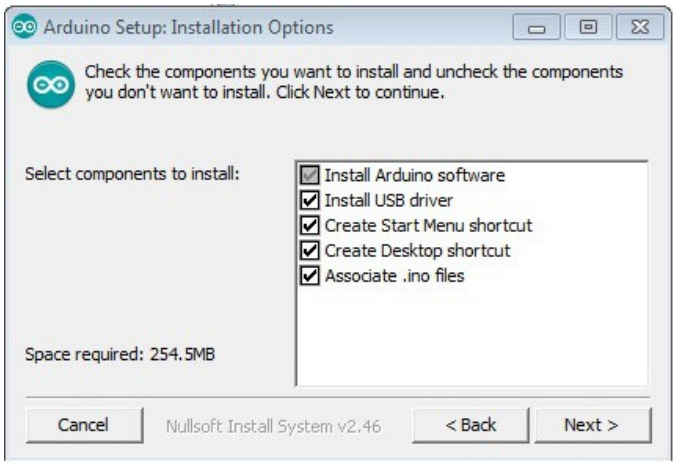

Figura 1.6

Mantenemos todos los componentes seleccionados y pulsamos "Next". Obtendremos la imagen de la Figura 1.7. Seleccionamos el directorio de instalación, no es necesario que esté creado, y pulsamos "Install".

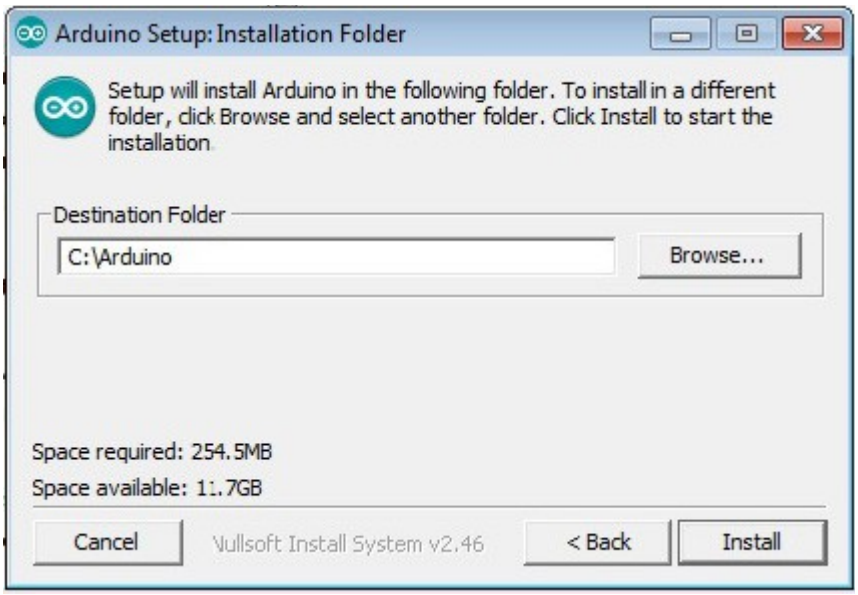

Figura 1.7

Aparecerá una nueva pantalla como la que se muestra en la Figura 1.8. Aquí decidimos si instalar el driver USB para la conexión del dispositivo arduino que estemos utilizando. "Check" *Siempre confiar en el software de "Arduino LLC"*. Pulsamos "instalar" y esperamos a que aparezca el icono

Figura 1.8

Entonces, en la Figura 1.9 pulsar "Close", finalizando la instalación de la plataforma IDE de Arduino.

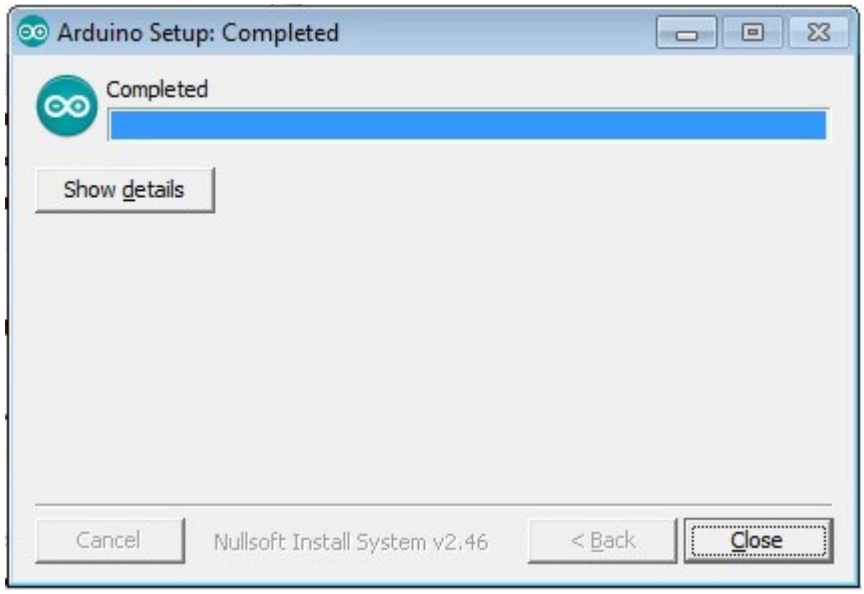

Figura 1.9

1.4 Conexión Placa Arduino. Primeros Pasos

Para trabajar con el entorno arduino, necesitamos una placa arduino y un cable USB como el que se muestra en la figura 1.1. Conectamos el cable a la placa y el puerto USB del PC. La primera vez, observaremos un mensaje en la parte inferior derecha de la barra de tareas de windows un mensaje como el que se muestra en la Figura 1.10.

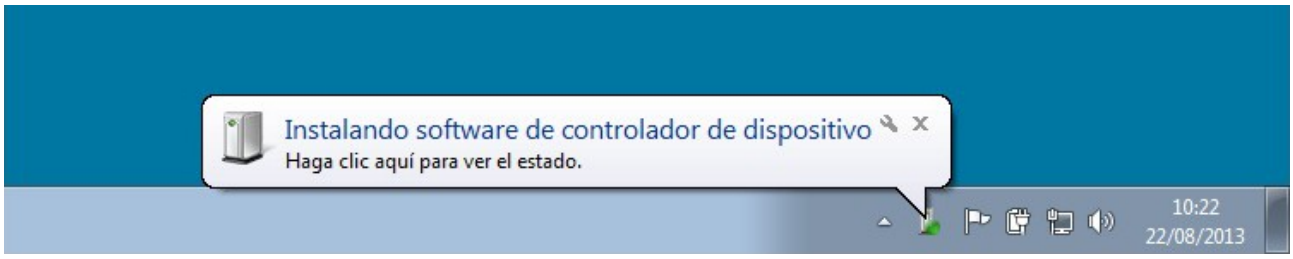

Figura 1.10

Después de un tiempo, si el driver se ha instalado con éxito, se mostrará un segundo mensaje (Figura 1.11) indicando que este ha sido instalado con éxito y el puerto COM al que se ha asociado la placa Arduino.

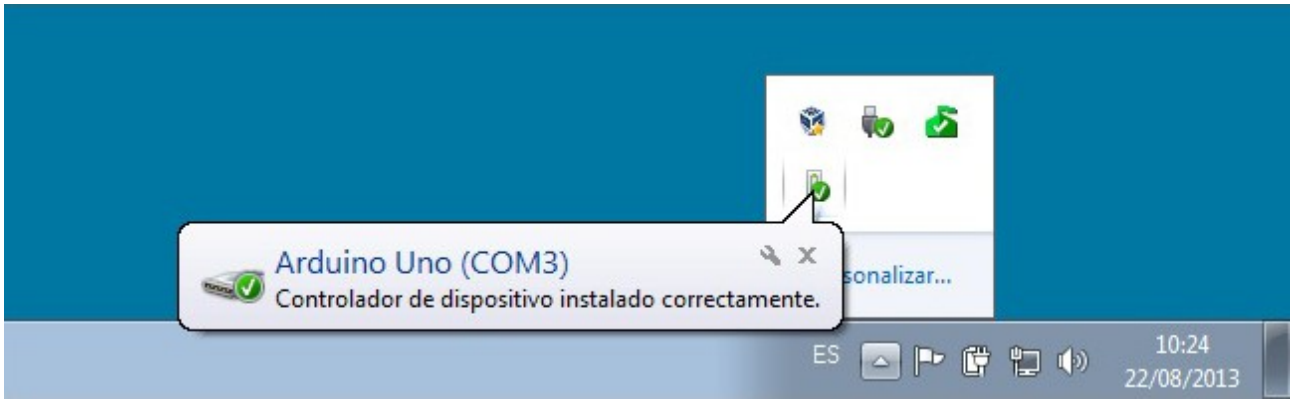

Figura 1.11

Una vez conectada la placa Arduino al PC, podemos iniciar el entorno IDE de arduino haciendo Dbl-Click en el icono y obtenemos el la ventana mostrada en la Figura 1.12

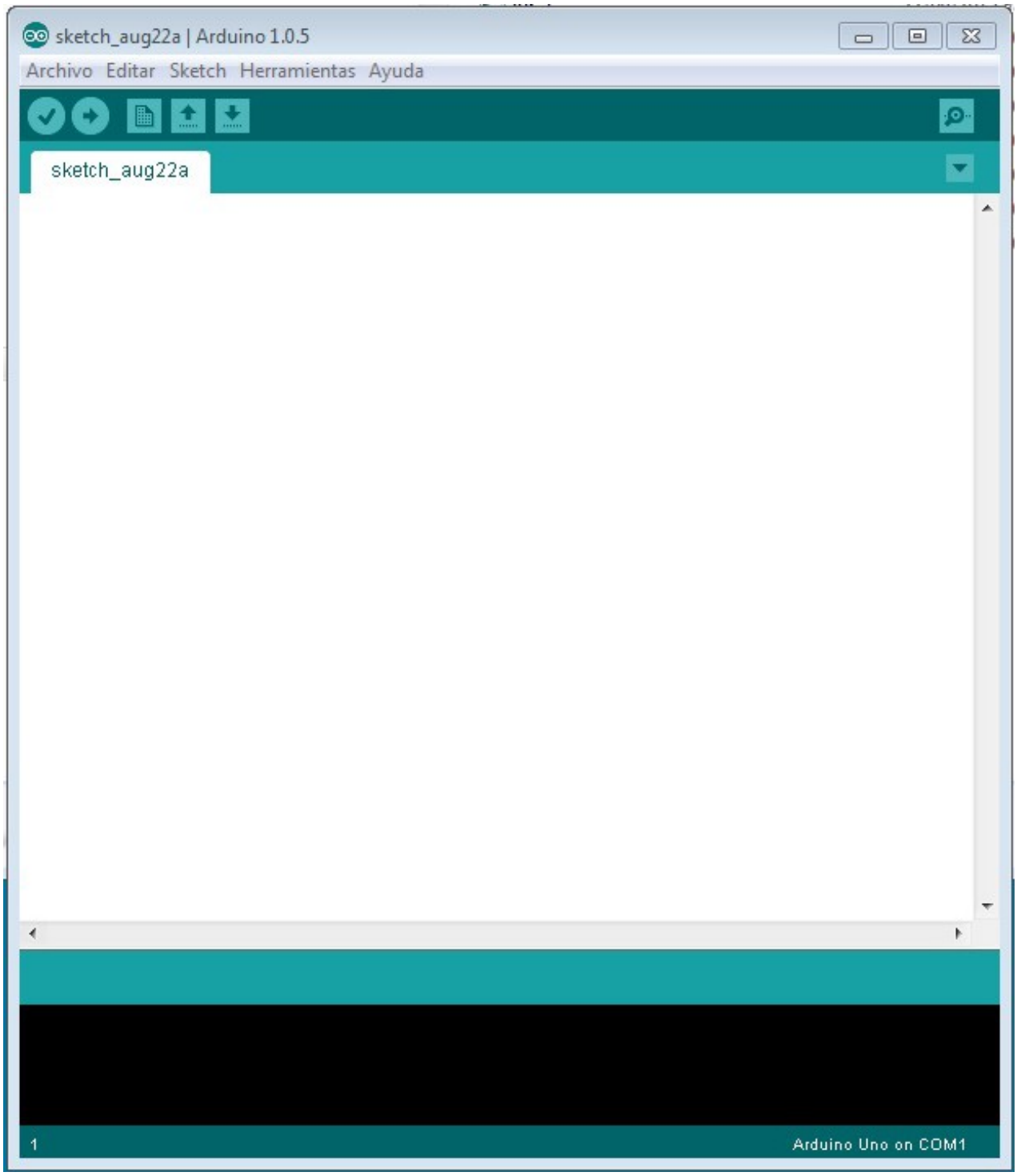

Figura 1.12

Cargamos una rutina de ejemplo. En el entorno Arduino lo llamamos “Sketch” pulsando

“Archivo → Ejemplos → Basics → Blink”.

Como se muestra en la Figura 1.13

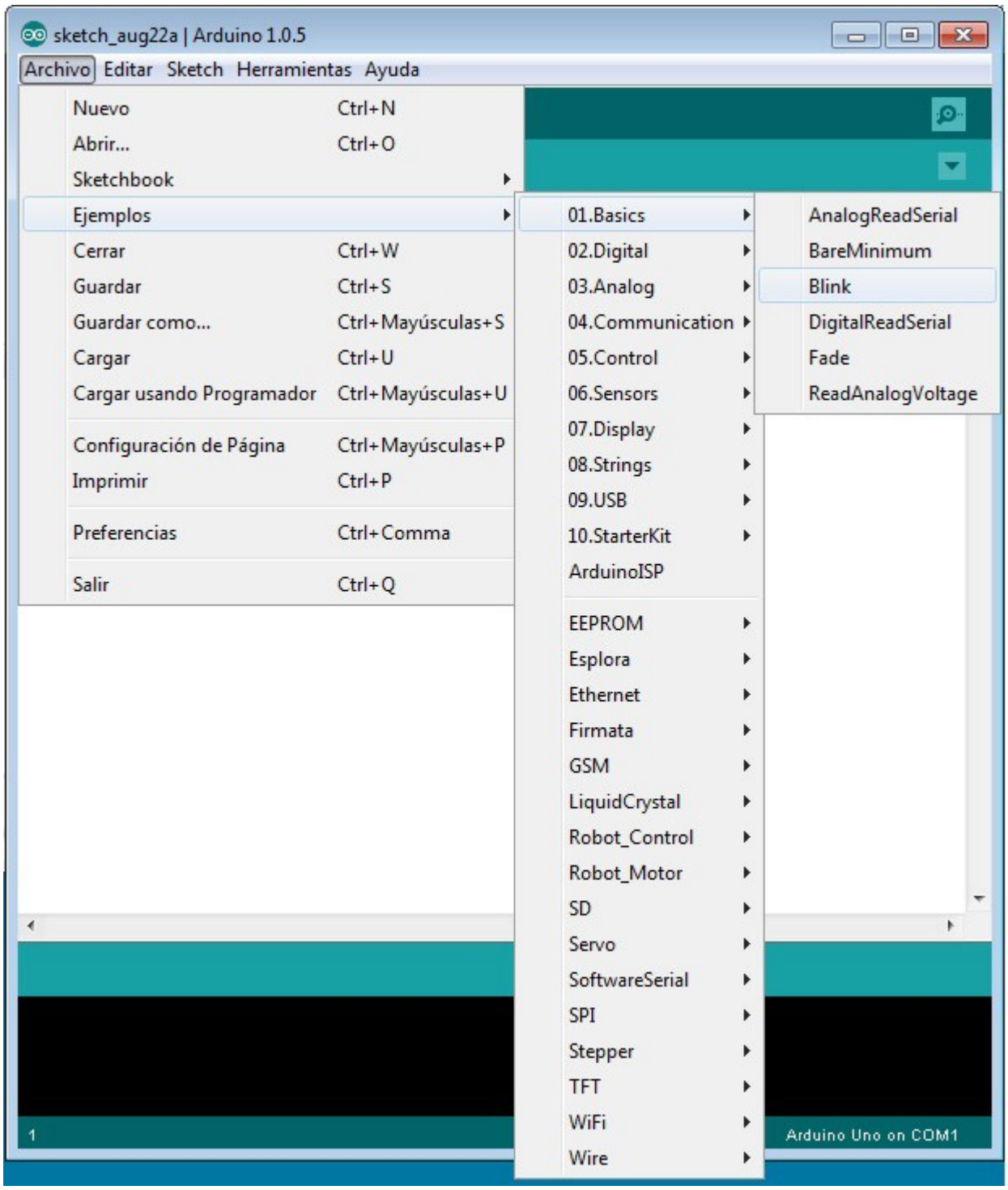

Figura 1.13

En la nueva pantalla se mostrará el entorno IDE con una nueva pestaña (“Blink”) donde se integrará

el fuente del scketch (Figura 1.14).

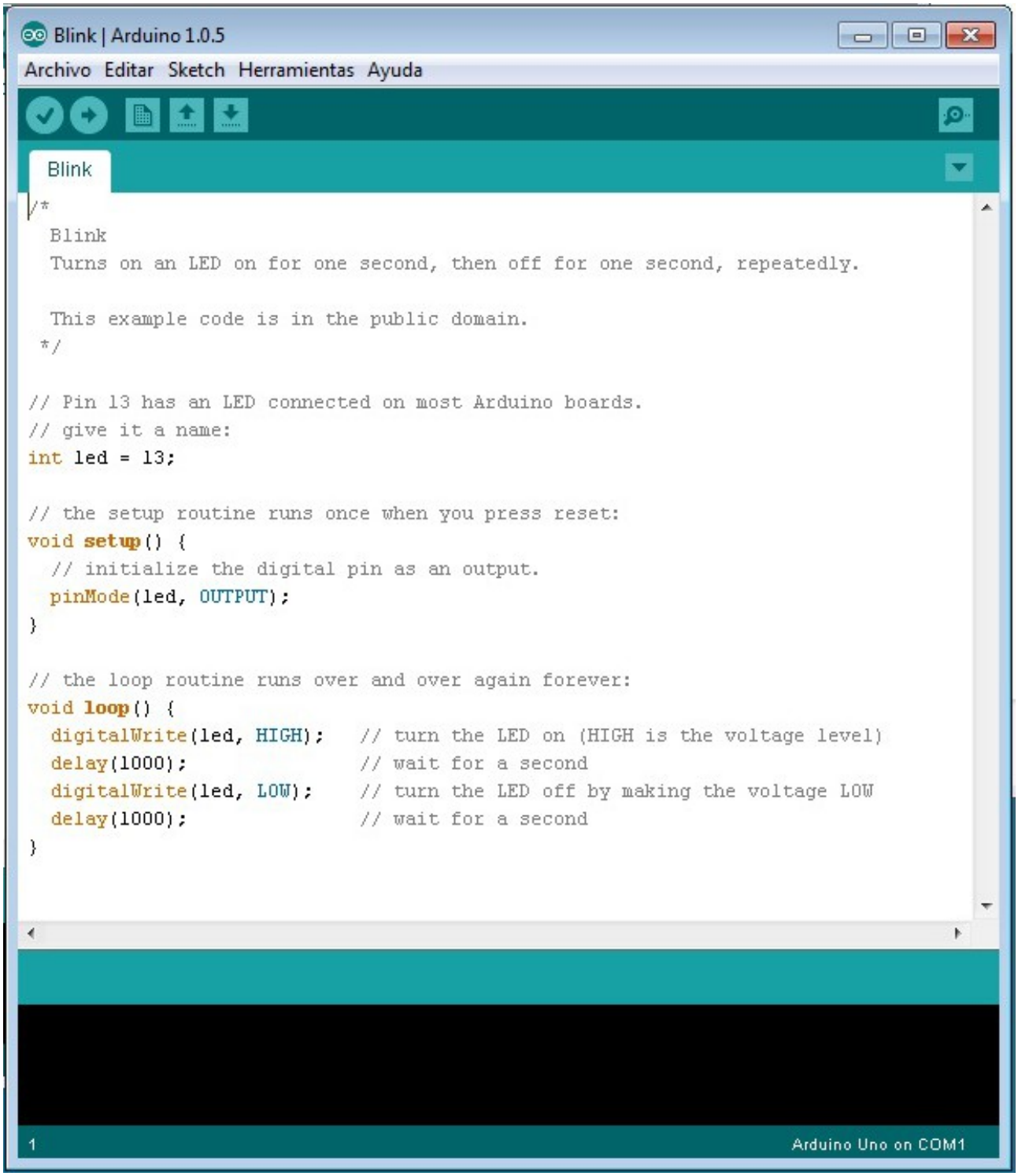

Figura 1.14

La placa arduino con la que deseamos trabajar, debe ser seleccionada en el entorno IDE pulsando:

"Herramientas → Tarjeta → Arduino UNO"

como se indica en la figura 1.15

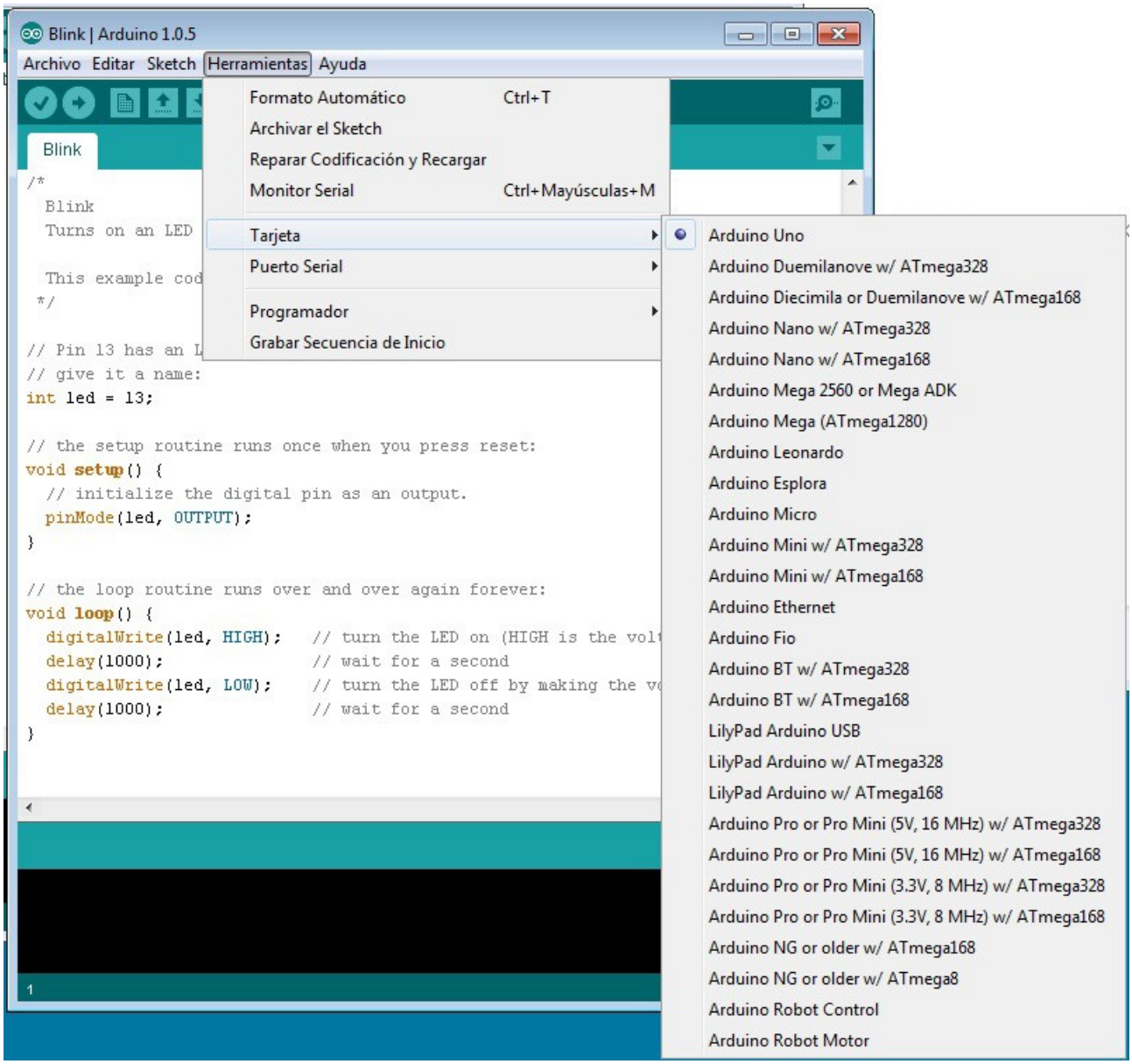

Figura 1.15

Una vez seleccionada la placa arduino que va ser usada, se debe seleccionar el dispositivo serie a través del que el PC host se va a comunicar con la misma. Con anterioridad, el sistema nos ha indicado el dispositivo serie a través del cual la placa arduino se conecta (Figura 1.11). Si no se recuerda este dispositivo serie, se puede consultar en el administrador de dispositivos de windows, ta y como se muestra en la figura 1.16, podemos ver que la placa arduino Uno ha sido conectada al puerto serie COM3.

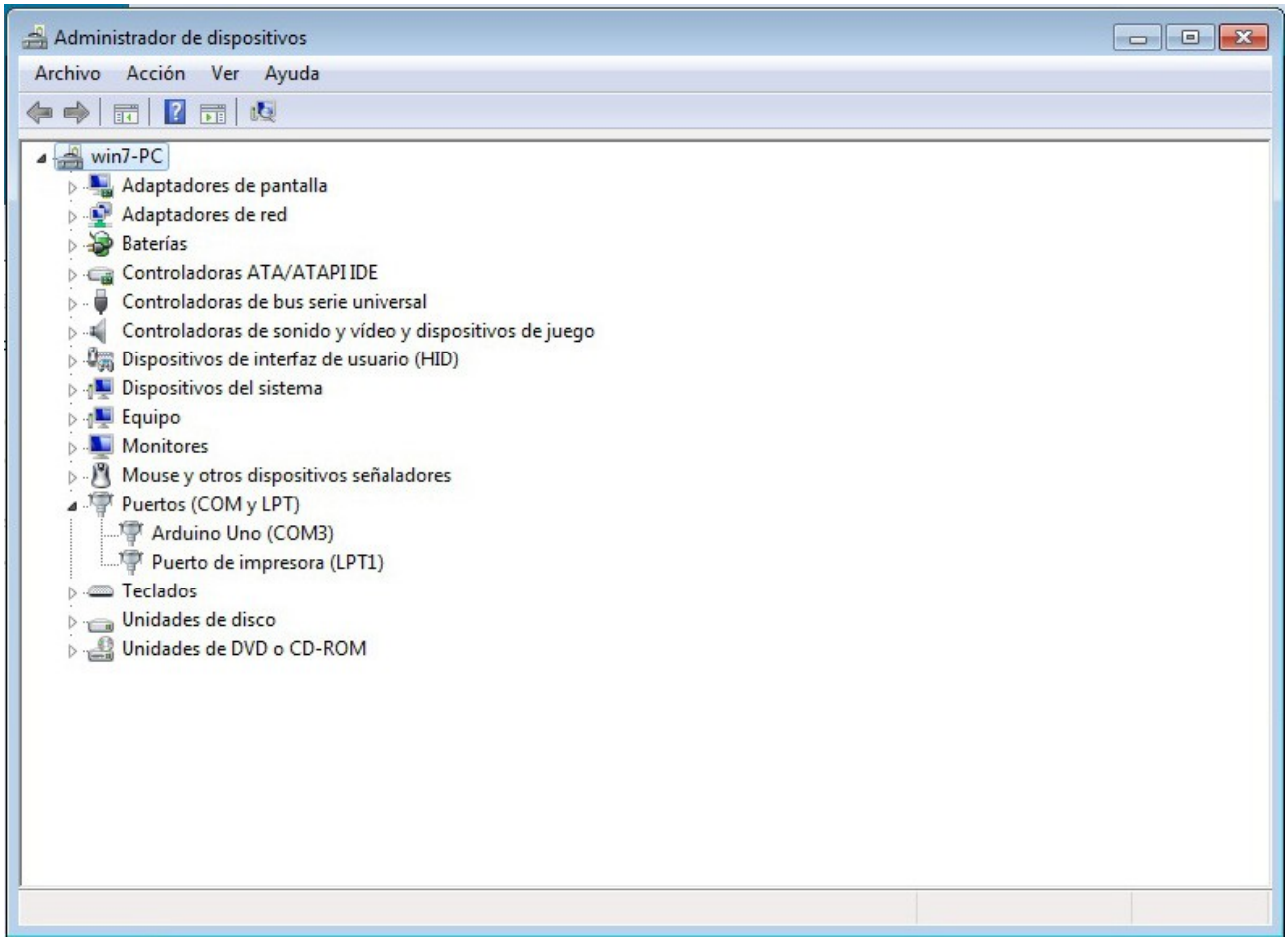

Figura 1.16

Una vez se conoce el puerto serie al que se conecta la placa, este debe ser seleccionado en el entorno IDE pulsando

"Herramientas → Puerto Serial → COM3"

como se muestra en la figura 1.17

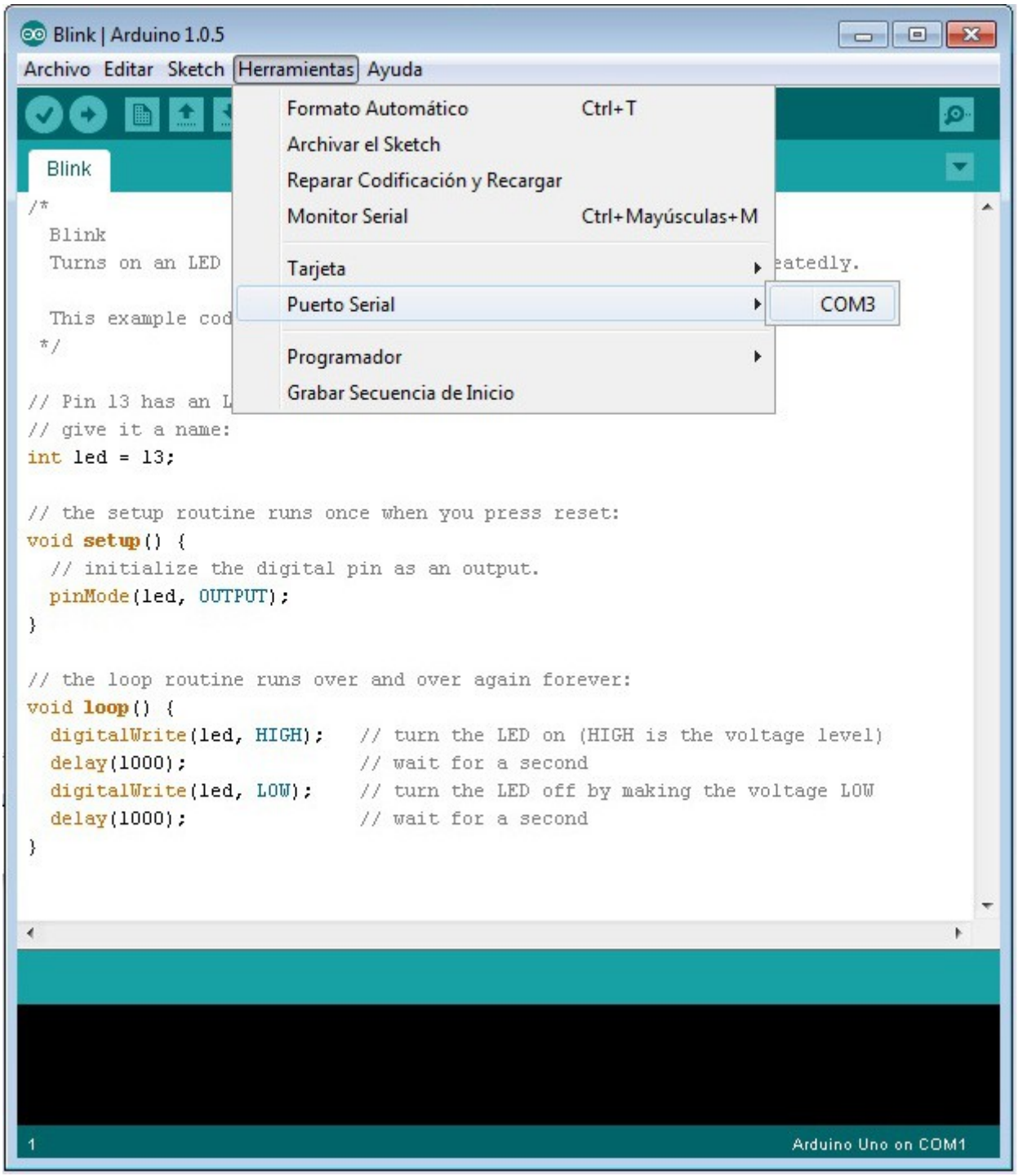

Figura 1.17

1.5 Entorno IDE Arduino

El entorno IDE de arduino ya ha sido mostrado con anterioridad en distintas imágenes, como por ejemplo en la figura 1.12, por lo que dedicaremos algún tiempo a describir algunos de los aspectos más importantes, que permitirá de forma rápida iniciar la programación de las placas arduino . En la figura 1.18 se puede ver algunas indicaciones para la barra de herramientas.

- Verifica el código buscando errores de sintaxis

- Cargar el código en la placa arduino seleccionada.

- Crear un nuevo Sketch

- Muestra un menú para abrir un Sketch almacenado

- Muestra una ventana para almacenar el Sketch activo

- Abre una ventana donde se muestran los mensajes enviados desde la placa arduino a través del puerto serie. Para enviar datos, se escribe sobre el espacio reservado en la parte superior y se pulsa el botón “enviar” o “Enter”. Seleccionar también la velocidad de transmisión en el menú desplegable con el mismo valor usado en su sketch para iniciar el puerto serie **Serial.begin** (figura 1.19).

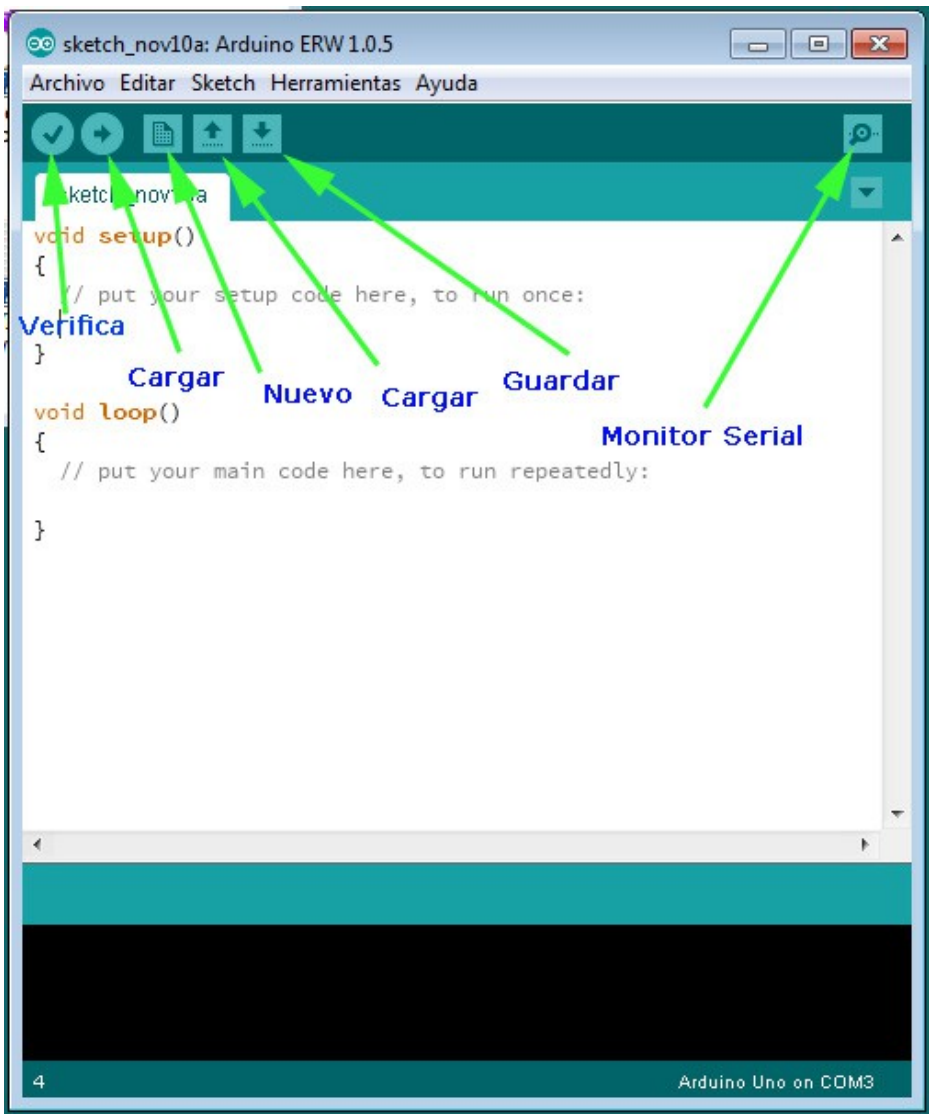

Figura 1.18: Barra Herramienta entorno IDE Arduino

Figura 1.19: Serial Monitor Arduino

En cuanto a la barra de menús, es conveniente prestar una atención especial a algunas opciones que puede ayudar al manejo de proyectos y los entornos con los que estemos trabajando.

1.5.1.1 Sketch

- *Importar Librería*: Añade una librería a la aplicación proporcionándole una nueva funcionalidad, para ello se incluye la linea #include … al inicio del código. Esto incrementa el tamaño de la aplicación que se va a cargar en la placa arduino. A partir de la versión 1.0.5 y siguientes, se puede importar una librería desde u fichero .zip.
- *Verificar/Compilar*: comprueba el sketch buscando errores.
- *Mostrar la Carpeta Sketch*: Abre la carpeta del actual sketch.
- *Agregar Archivo...*: Añade ficheros al sketch desde la carpeta actual del sketch. Este nuevo fichero se mostrará en una nueva lengüeta en la ventana del sketch. Los ficheros se pueden quitar usando el menú de la lengüeta.

1.5.1.2 Herramientas

- *Formato Automático*: Da formato al código, usando la estética como criterio, por ejemplo, usando tabulaciones entre la apertura y el cierre de llaves.
- *Tarjeta*: Selecciona la placa arduino que se está usando.
- *Puerto Serie*: Seleccionar el puerto serie que está usando la placa arduino conectada. Se actualizará de forma automática cada vez que se abra el menú Herramientas.
- *Grabar Secuencia de Inicio (Bootloader)*: Permite grabar un gestor de arranque (bootloader) dentro del microcontrolador de la placa arduino. Aunque no es un requisito para el normal funcionamiento de la placa Arduino, será útil si se compra un nuevo ATmega (el cual viene normalmente sin gestor de arranque). Asegúrese de haber seleccionado la placa correcta en el menú **Tarjeta** antes de grabar el *bootloader*. Cuando use AVR ISP, tendrá que seleccionar en el menú **Puerto Serie** el puerto correspondiente.

1.5.1.3 Sketchbook (Librería de Sketch)

El entorno Arduino usa para el almacenamiento de sus rutinas (sketch) un directorio al que se llama sketchbook. Esta carpeta puede abrirse desde el menú **Fichero → Sketchbook**, y puede ser modificado desde el apartado **Fichero → Preferencias**.

1.5.1.4 Uploading (Volcado)

Para descargar los programas en la placa arduino correspondiente, se selecciona la tarjeta y el puerto serie al que se conecta (**Herramientas → Tarjeta; Herramientas → Puerto Serie**) y se pulsa el botón de cargar o seleccionamos **Fichero → Cargar**. En las nuevas placas arduino, no es necesario pulsar el botón de reset, mientras que en las placas antiguas si. Durante la descarga del software en las placas, se mostrará un parpadeo de los led's RX y TX. Al finalizar la descarga, el entorno arduino mostrará un mensaje, o mostrará un mensaje de error.

En la descarga de las rutinas, se usa el bootloader arduino, un programa instalado en el microcontrolador de la placa, que permite el volcado del código sin usar un hardware adicional. Este programa está activo durante un pequeño periodo de tiempo al iniciar la placa y a continuación

se inicia la rutina que más recientemente de hubiese cargado en la placa. El bootloader de arduino, produce un parpadeo en el LED de la placa (pin 13).

1.6 Estructura de un programa arduino

Todos los programas (sketch) arduino tienen una estructura muy simple y se compone de dos partes bien diferenciadas:

```
void setup() {

  estamentos;

}

void loop () {

  estamentos;

}
```

donde

setup()

Es la rutina encargada de iniciar la configuración del sistema de acuerdo a las necesidades de la aplicación y la primera en ejecutarse cuando se inicia el sketch, por lo que en ella se debe iniciar la variables usadas, así como la configuración inicial del hardware del microcontrolador (pines, puerto serie, timers, etc).

loop()

Se ejecutará cíclicamente y recogerá el cuerpo de la aplicación. Es el centro del programa y de el partirán todas las funciones que serán ejecutadas.

Estas dos funciones serán necesarias para que funcione cualquier sketch en arduino.

A partir de aquí, el uso de funciones, variables, estructuras de control,… es similar a como se usa en C/C++, por lo que profundizar en la programación con dispositivos arduino, usando el entorno de desarrollo arduino, se podría realizar usando cualquier manual de C/C++, aunque las librerías arduino definen una serie de macros

1.7 Variables

El desarrollo de cualquier programa, sea cual sea el lenguaje de programación usado, necesita de valores, datos, que pueden ser modificados a medida que se ejecuta. Su buen uso es lo que de alguna manera dota al sistema de la adaptabilidad necesaria al entorno/aplicación, y es a lo que llamamos **variable**, en definitiva, un elemento útil para nombrar, almacenar un valor susceptible de ser usado con posterioridad. Los datos manejados, pueden ser muy diversos, por lo que deben ser declaradas con anterioridad a su uso, indicando el tipo de datos que van a contener. Habitualmente,

las variables son declaras usando un modificador que indica el tipo de datos que va a contener, junto con el nombre, que debería ser descriptivo del uso que va a tener, para facilitar y clarificar la lectura del código.

int valorInterruptor; (1.1)

En el caso anterior (ec.1.1), se muestra la definición de una variable de tipo entero que será utilizada para almacenar el valor de algún tipo de interruptor. Las variables mas utilizadas en C (Arduino), se pueden ver en la tabla 1.2

Tipo	Descripción
char	Tipo básico alfanumérico, puede contener un carácter, un dígito numérico o un signo de puntuación
int	Almacena números enteros, dentro de los límites de su tamaño, que depende de la plataforma que estemos utilizando.
float	Almaceno números en formato de punto flotante
bool	Sólo pueden tomar valores **verdadero o falso**
double	Almacena números en punto flotante igual que float, pero con una precisión mayor.
Void	Tipo especial que indica la ausencia de tipo

Tabla 1.2: Tipos fundamentales de Variables

Las variables pueden ser declaradas en distintos lugares dentro del código, lo cual nos definirá las partes del programa desde donde pueden ser utilizadas (ámbito de aplicación). Las variables pueden ser declaradas en dos posiciones fundamentales, al principio del programa, antes de de la ejecución de cualquier función, fuera de las funciones que forman parte del programa, en cuyo caso, son **globales,** es decir, pueden ser utilizadas en cualquier parte del programa, o dentro de una función (bloque), **locales**, en cuyo caso, sólo podrán ser utilizadas dentro de esa función (bloque).

1.8 Operadores

Con las variables, podemos hacer distintas operaciones usando modificadores a los que llamamos operadores, con el uso de distintos operadores obtendremos distintos resultados.

int x = 5;

int y = 4;

int z = x + y ; z tomará el valor 9

En las instancias anteriores, podemos ver la definición de las variables enteras x, y, su inicialización, y posteriormente la definición de la variable z y su inicialización usando el operador suma (+).

En C podemos en podemos encontrar una gran cantidad de operadores distribuidos en varias

categorías: aritméticos, relacionales, lógicos, incremento y decremento, asignación y manejo de bits.

SÍMBOLO	DESCRIPCIÓN	EJEMPLO
+	SUMA	a + b
-	RESTA	a - b
*	MULTIPLICACIÓN	a * b
/	DIVISIÓN	a / b
%	MÓDULO	a % b
-	SIGNO	- a

Tabla 1.3: Operadores aritméticos

El operador módulo (%) se utiliza para calcular el resto del cociente entre dos ENTEROS , y NO puede ser aplicado a variables del tipo float ó double.

SÍMBOLO	DESCRIPCIÓN	EJEMPLO
<	MENOR QUE	a < b
>	MAYOR QUE	a > b
<=	MENOR O IGUAL QUE	a <= b
>=	MAYOR O IGUAL QUE	a >= b
==	IGUAL QUE	a == b
!=	DISTINTO QUE	a != b

Tabla 1.4: Operadores Relacionales

Uno de los errores más comunes es confundir el operador relacional IGUAL QUE (==) con el de asignación IGUAL A (=). La expresión a=b copia el valor de b en a, mientras que a = = b retorna un cero , si a es distinto de b ó un número distinto de cero si son iguales.

SÍMBOLO	DESCRIPCIÓN	EJEMPLO
&&	Y (AND)	a && b
||	O (OR)	a || b
!	NEGACIÓN (NOT)	!a

Tabla 1.5: Operadores Lógicos

Los resultados de la operaciones lógicas siempre adoptan los valores CIERTO ó FALSO. La evaluación de las operaciones lógicas se realiza de izquierda a derecha y se interrumpe cuando se ha asegurado el resultado. El operador NEGACION invierte el sentido lógico de las operaciones

SÍMBOLO	DESCRIPCIÓN	EJEMPLO
++	INCREMENTO	++a ó a++
--	DECREMENTO	--a ó a--

Tabla 1.6: Operadores Incremento y Decremento

El sentido de los operadores anteriores es:

a = a + 1 ;

a++ ;

ambas operaciones tienen una acción idéntica, de la misma forma que

a = a - 1 ;

a-- ;

es decir incrementa y decrementa a la variable en una unidad. Poner el operador antes o después de la variable hace que este sea usado antes o después, respectivamente, de usar la variable.

SÍMBOLO	DESCRIPCIÓN	EJEMPLO
=	IGUAL A	a = b
op=	PSEUDOCÓDIGO	a += b
=?:	ASIGNACIÓN CONDCIONAL	a = (c>b)?d:e

Tabla 1.7: Operadores Asignación

El funcionamiento del operador condicional es el siguiente :

destino = (operación relacional ó logica) ? (fuente 1) : (fuente 2) ;

como resultado de la operación condicional se asignará a destino el valor de fuente 1 ó 2 . Caso de ser CIERTA tendremos destino = fuente 1, caso de ser FALSO, destino = fuente 2.

SÍMBOLO	DESCRIPCIÓN	EJEMPLO
&	Y ó AND (bit a bit)	a & b
|	O ú OR INCLUSIVA	a | b
^	O ú OR EXCLUSIVA	a ^ b
<<	ROT. IZQUIERDA	a << b
>>	ROT. DERECHA	a >> b
~	COMPLEMENTO A UNO	~a

Tabla 1.8: Operadores Manejo de Bits

como ejemplo de utilización de los operadores de manejo de bits podemos ver el ejemplo siguiente:

```
bits                    decimal      hexadecimal
0 1 0 1 0 1 0 1         85           55
  &                     &            &
0 1 0 0 0 1 1 1         71           47
-------------------------  -------      -------
0 1 0 0 0 1 0 1         69           45
```

PRECEDENCIA	OPERADORES	ASOCIATIVIDAD
0	()[]	izq. a derecha
1	sizeof (tipo) ! ~ ++ -- signo* &	derecha a izq.
2	* / %	izq. a derecha
3	+ -	izq. a derecha
4	>	izq. a derecha
5	>=	izq. a derecha
6	== !=	izq. a derecha
7	&	izq. a derecha
8	^	izq. a derecha
9	|	izq. a derecha
10	&&	izq. a derecha
11	||	izq. a derecha
12	?:	derecha a izq.
13	= += -= *= etc	derecha a izq.

Tabla 1.9: Precedencia de los operadores

1.9 Control flujo de programa.

Las variables junto con los operadores que hemos visto anteriormente, son usados para obtener nuevos valores que pueden influir en nuestras expectativas a la hora de ejecutar programas. En este sentido, cualquier lenguaje de programación dispone de una serie de estamentos con los que poder controlar el flujo de ejecución en función de los resultados de ciertas operaciones.

1.9.1 If … else

If es un estamento usado como decisión, ejecución de distintas partes de código en función de una condición:

```
if (variable ?? valor)
{
      marco de ejecución
}
```

Si se cumple la condición representada por *(variable ?? valor),* cuyo resultado tendrá que ser de tipo "booleano", VERDADERO o FALSO, se ejecutará el código representado por "marco de ejecución". Al final del ejemplo anterior se puede añadir la sentencia ***"else"*** que indicara lo porción de código que deberá ejecutarse, caso de no cumplirse la condición inicial.

```
if (variable ?? valor)
{
      marco de ejecución
}
else{
      instrucciones
}
```

Sucesivamente, ***else*** puede ir seguida de una nueva sentencia ***if … else*** si queremos implementar una nueva sentencia excluyente y seguir así indefinidamente (anidamiento).

1.9.2 Repeticiones

Como hemos dicho, la sentencia **if … else** tiene la capacidad de decidir si una porción de código es ejecutado o no, y eso se realizará una sola vez. Sin embargo, en determinadas circunstancias es necesario que una porción de programa sea ejecutada un número determinado de veces, es decir, se necesita que una porción de código sea repetido, lo cual no lleva al enunciado de esta sección.

1.9.2.1 for(…)

La sentencia for es usada para realizar la repetición de una porción de código, un determinado número de veces. La declaración de la sentencia es como sigue:

```
for(inicialización;condición;expresión)
{
      Marco de ejecución
}
```

La inicialización de una variable local se produce una sola vez, con cada bucle se aplica la expresión y a continuación se comprueba la condición. Si la condición sigue cumpliéndose, se vuelve a ejecutar el marco de ejecución. Cuando la condición no se cumpla, finalizaremos el bucle.

En el ejemplo que sigue, inicializamos una variable de tipo entero, *i*, establecemos la condición de repetición *(i < 20)*, e indicamos la expresión de actualización de la variable *(i++)*, se incrementa con cada repetición

```
for(int i = 0; i<20; i++)
{
      Marco de ejecución
}
```

Cualquiera de los tres elementos de cabecera puede omitirse, aunque el punto y coma es obligatorio. También las declaraciones de inicialización, condición y expresión puede ser cualquier estamento válido en lenguaje C sin relación con las variables declaradas.

1.9.2.2 While(…)

A diferencia del bucle **for**, donde de alguna manera el número de iteraciones esta limitado, el bucle **while** el número de iteraciones no están definidas, y se producirá mientras se cumpla la condición indicada en el paréntesis.

```
while (variable ?? valor)
{
      marco de ejecución
}
```

esta condición podrá cambiar en función de los resultados obtenidos durante la ejecución del ***marco de ejecución***.

1.9.2.3 Do … while

El bucle ***do … while*** funciona igual que el bucle ***while***, con la diferencia que la condición de repetición del bucle se realiza al final. Esto implica, que el código del marco de ejecución se repetirá al menos una vez.

```
do
{
      marco de ejecución
}while (variable ?? valor)
```

1.10 Pinceladas API Arduino.

Como ya sabemos, Arduino es una plataforma hardware basada en microcontroladores Atmel AVR con puertos de entrada/salida y que por su sencillez en el uso, por la rapidez con la que se puede hacer uso de sus recursos y su bajo coste, la hace ideal para entornos de formación en la que el objetivo principal del docente no es la formación en el lenguaje de programación.

Ya hemos visto también, que el lenguaje de programación usado en arduino es el C/C++, aunque salpicado de algunas particularidades que forman parte de las librerías Arduino, que facilitan la labor al usuario.

1.10.1 pinMode(pin, mode)

En la descripción de la plataforma arduino UNO (Figura 1.1) que se ha realizado con anterioridad, entre otras funcionalidades, hemos mostrado (Figura 1.3) la disponibilidad de unas entradas /salidas digitales. La utilización de esta instrucción permite configurar de forma individual cada uno de los pines como salida entrada.

```
Ejm:  pinMode(13, OUTPUT);
      pinMode(10, INPUT);
```

configura el pin 13 como salida y el pin 10 como entrada.

1.10.2 digitalWrite(pin, valor)

Cuando un pin digital ha sido declarado como salida, significa que a través de el podemos presentar distintos valores digitales “1” (5 V.) ó “0” (0 V.) a lo largo del del tiempo. Para ello usamos la instrucción ***digitalWrite*** de la siguiente forma:

```
Ej:   digitalWrite(13, HIGH);
      digitalWrite(13, LOW);
```

según queramos que en el pin 13 aparezca respectivamente, un valor alto (5 V.) o un valor bajo (0 V.).

1.10.3 digitalRead(pin)

Por el contrario, cuando un pin ha sido declarado como entrada, podremos leer su estado, alto (5 V.) o bajo (0 V.) por lo que cambio esternos pueden ser usados por la aplicación con la que estemos trabajando, con sólo usar digitalRead:

```
Ej:   val = digitalRead(10);
```

1.10.4 analogRead(pin)

El dispositivo arduino utilizado como referencia para este texto, dispone de 6 estradas analógicas (Figura 1.4), donde podemos leer un valor analógico desde el exterior. Estas entradas, están equipadas con convertidores A/D (Analógico/Digitales) de 10 bits. Esto significa que convertirá tensiones entre 0 y 5 voltios a un número entero entre 0 y 1023. Esto proporciona una resolución en la lectura de: 5 voltios / 1024 unidades, es decir, 0.0049 voltios (4.9mV) por unidad. El rango de entrada puede ser cambiado usando la función ***analogReference()***.

```
Ej: val = analogRead(3);
```

1.10.5 analogWrite(pin, value)

La Figura 1.3 muestra los pines usados por la plataforma como E/S digitales, si observamos con atención, se dispone de 6 pines (3, 5, 6, 9, 10, 11) marcados también como PWM, pines usados por la instrucción ***analogWrite*** (escribe un valor analógico del tipo (PWM) en un pin.). Después de llamar a la función ***analogWrite()***, el pin generará una onda cuadrada estable con el ciclo de trabajo especificado hasta que se vuelva a llamar a la función ***analogWrite()*** (o una llamada a las funciones ***digitalRead() o digitalWrite()*** en el mismo pin). La frecuencia de la señal PWM será de aproximadamente 490 Hz. los valores de ***analogRead*** van desde 0 a 1023 y los valores de ***analogWrite*** van desde 0 a 255, 0 (siempre apagado) y 255 (siempre encendido).

```
Ej:  val =  analogRead(analogPin);
            analogWrite(ledPin, val / 4);
```

1.10.6 Comunicación Serie

Se utiliza para la comunicación entre la placa Arduino y un ordenador u otros dispositivos. En la plataforma usada de ejemplo (Figura 1.20) se muestran los pines digitales donde se implementa el puerto serie (0-RX y 1-TX). Otras plataformas arduino pueden tener más de un puerto serie.

```
Serial.begin(speed)
```

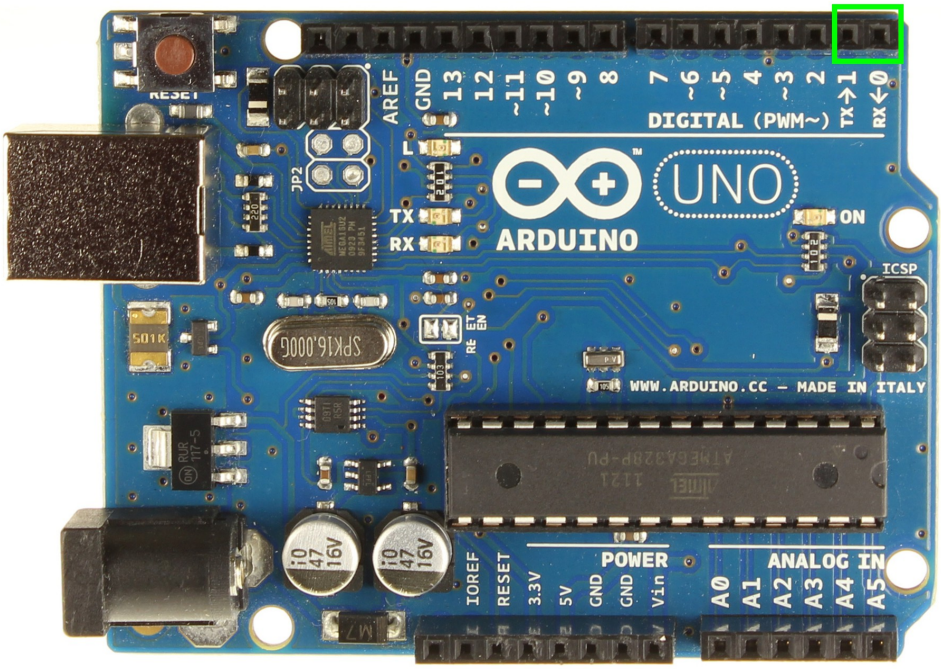

Figura 1.20: Puerto Serie

Configura los pines 0 y 1 para ser usado como puerto serie y establece la velocidad de datos en bits por segundo (baudios) para la transmisión de datos en serie. Los valores comúnmente usados son: 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600 o 115200.

```
Ej:   Serial.begin(9600);
      Serial.read()
```

Lee los datos entrantes del puerto serie.

```
Ej:   Byte = Serial.read();
```

por otra parte:

Serial.print(val,[format]),

imprime los datos al puerto serie como texto ASCII.

val: el valor a imprimir - de cualquier tipo

format: especifica la base (formato) a usar; los valores permitidos son, BYTE, BIN (binarios o base 2), OCT (octales o base 8), DEC (decimales o base 10), HEX (hexadecimales o base 16). Para números de coma flotante, este parámetro especifica el número de posiciones decimales a usar.

```
Ej:   Serial.print(78) imprime "78"
      Serial.print('N') imprime "N"
      Serial.print(78, BYTE) imprime "N"
      Serial.print(78, DEC) imprime "78"
      Serial.println(1.23456, 0) imprime "1.23"
      Serial.println(1.23456, 2) imprime "1.2
```

Serial.println(val,[format])

Imprime los datos al puerto serie como texto ASCII seguido de un retorno de carro (ASCII 13, o '\r') y un carácter de avance de línea (ASCII 10, o '\n').

```
Ej:   Serial.println(analogValue);        // imprime como ASCII decimal
      Serial.println(analogValue, HEX);   // imprime como ASCII hexadecimal
```

Serial.available()

Devuelve el número de bytes (caracteres) disponibles para ser leídos por el puerto serie. Se refiere a datos ya recibidos y disponibles en el buffer de recepción del puerto (que tiene una capacidad de 128 bytes).

```
if (Serial.available() > 0)
{
      //realiza la lectura del puerto serie
}
```

1.10.7 Tiempo

delay(ms)

detiene la ejecución del programa en este punto un número de milisegundos, especificado en

el parámetros *ms.*

```
Ej:   delay(1000); //detiene el program1 Seg.
```

millis()

Esta función devuelve el número de milisegundos que han transcurrido desde que comenzó a ejecutarse el programa actual. Este número se reinicia cada 50 días.

delayMicroseconds(us)

detiene la ejecución del programa en este punto un número de microsegundos indicados por el parámetro **us**.

```
Ej:   delayMicroseconds(100); Pausa el programa 100 microsegundos
```

micros()

Esta función devuelve el número de microsegundos desde que se comenzó a ejecutar el program actual. Este número se reinicia cada 70 minutos aproximadamente.

1.10.8 Matemáticas

min(x, y)

Calcula el mínimo de los dos números (x, y)

max(x, y)

Calcula el máximo de los dos números (x, y)

constrain(x, a, b)

restringe el valor de un número a un rango, es decir, esta función devuelve:

```
x: si si x está entre a y b
a: si x es menor que a
b: si x es mayor que b
```

1.10.9 Números aleatorios

randomSeed(seed)

Inicializa un generador de números pseudo-aleatórios. Esto hace que en la generación de secuencias aleatorias se seleccione un punto de partida arbitrario. Las secuencia aleatorias generadas usando la función ***random()***, aunque muy largar, son siempre la misma. Si es muy importante que las secuencias generadas mediante la función ***random()*** sean diferentes, antes de esta función, deberíamos usar la función ***randomSeed()*** para inicializar el generador de números aleatorios, usando como parámetro un valor quasi-aleatorio, como podría ser el valor proporcionado por la función ***analogRead()*** que lea un pin no conectado, en cuyo caso, se leerá el ruido blanco

asociado a ese pin.

Por el contrario, a veces es útil usar secuencias pseudo-aleatorias que se repitan exactamente. En ese caso, deberíamos llamar a la función ***randomSeed()*** con un número fijo como parámetro antes de la llamada a ***random()***.

random()

genera un número pseudo-aleatorio.